TOPIC 03

Multiple Linear Regression

From one predictor to many: the model, dummy variables, backward elimination and the checks that keep it honest

WHAT YOU WILL LEARN

- Write the **Multiple Linear Regression (MLR)** equation and explain every symbol
- Explain what a coefficient means when **the other variables are held constant**, and why it can differ from a simple regression
- Turn a categorical column into **dummy variables**, and avoid the **dummy variable trap**
- Build, evaluate and read an MLR model with scikit-learn and statsmodels (R^2, adjusted R^2, t-tests, F-test)
- Run **backward elimination** and explain why p-values, adjusted R^2 and cross-validation can disagree
- Check the assumptions and **multicollinearity** (VIF), and spot perfect multicollinearity in a real dataset

Part A. The idea

1. From simple to multiple linear regression

Simple Linear Regression (SLR) predicts a number from **one** input. Real outcomes rarely depend on one thing. A company's profit depends on what it spends on digital marketing, on promotion, on research, and perhaps on where it operates. **Multiple Linear Regression (MLR) uses two or more independent variables to predict one dependent variable.**

DEFINITION **Multiple Linear Regression (MLR)**

A linear model that predicts one numerical **dependent variable (DV)** from **two or more independent variables (IVs)** by fitting a flat surface: a line is the one-input case, a plane the two-input case, and a hyperplane beyond that.

$$\hat{y} = b_0 + b_1 x_1 + b_2 x_2 + \cdots + b_n x_n$$

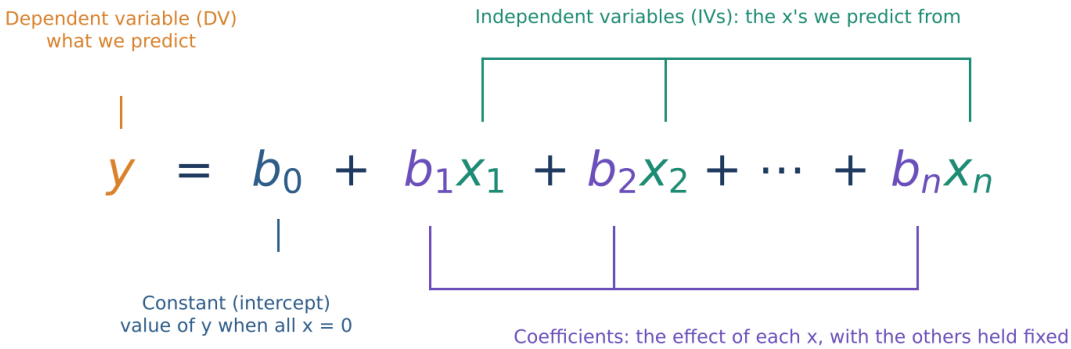


Figure 1. The MLR equation with the labels used in class: dependent variable, independent variables, constant and coefficients.

Symbol	Name	Meaning
y ($\hat{y}$)	Dependent variable (DV), also called target or output	The number we predict. $\hat{y}$ is the model's prediction, y the actual value
$x_1 \dots x_n$	Independent variables (IVs), also called features or predictors	The inputs. n is the number of them
b_0	Constant, also called intercept	The predicted y when every x is 0
$b_1 \dots b_n$	Coefficients, also called slopes or weights	How much the prediction changes when that x rises by one unit and **all the other x's stay fixed**

> **NOTE** **Same family as simple linear regression**
> With $n = 1$ the equation is the SLR line, $\hat{y} = b_0 + b_1x_1$. Nothing about the method changes: we still choose the coefficients that make the squared errors as small as possible. The difference is in **how we read the coefficients**, which is the next section.

2. What a coefficient means now

In SLR the slope is the change in y for one more unit of x. In MLR each coefficient carries an extra condition: it is the change in the average y for a one-unit increase in that x **while the other x's are held constant**. (This is the standard interpretation in regression texts, for example Penn State's STAT 462.)

> **WORKED EXAMPLE** **Reading a fitted model**
> On the investment data, the model gives **0.806** for DigitalMarketing. Reading: two companies that spend the same on Promotion and Research and are in the same state differ by about 0.806 in Profit for every 1 extra unit of DigitalMarketing spend.

> **COMMON MISTAKE** **A coefficient belongs to the whole model**
> Change the other variables in the model and a coefficient can change, sometimes a lot. It is not a fixed property of the variable on its own.

Here is a real case. Regressing Profit on **Research alone** gives a slope of **0.246** and a strong fit ($p < 0.001$). But Research and DigitalMarketing are correlated ($r = 0.72$): companies that spend more on one tend to spend more on the other. Once DigitalMarketing is in the model, most of what Research seemed to explain is already explained, and its coefficient drops to **0.027** ($p = 0.123$).

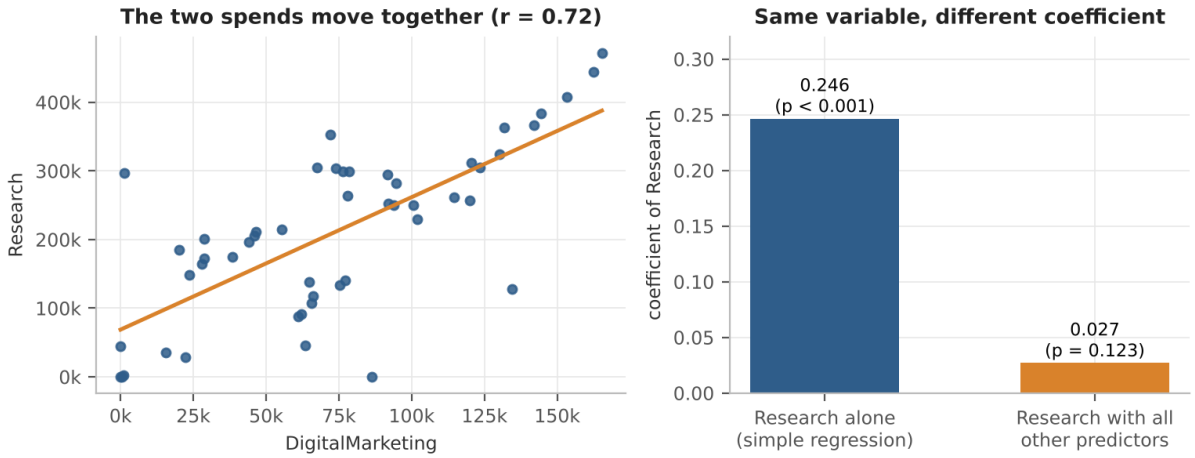


Figure 2. Left: the two spends move together. Right: the same variable, Research, gets a very different coefficient depending on what else is in the model.

KEY IDEA **Why this matters**

A simple regression asks *how does Profit change with Research?* An MLR asks *how does Profit change with Research once DigitalMarketing, Promotion and State are accounted for?* These are different questions, so the answers can differ.

3. The geometry

With one input the model is a line through a cloud of points. With two inputs it is a **plane** floating in three dimensions, and the errors are the vertical distances from each point to the plane. With more inputs we cannot draw it, but the idea is identical.

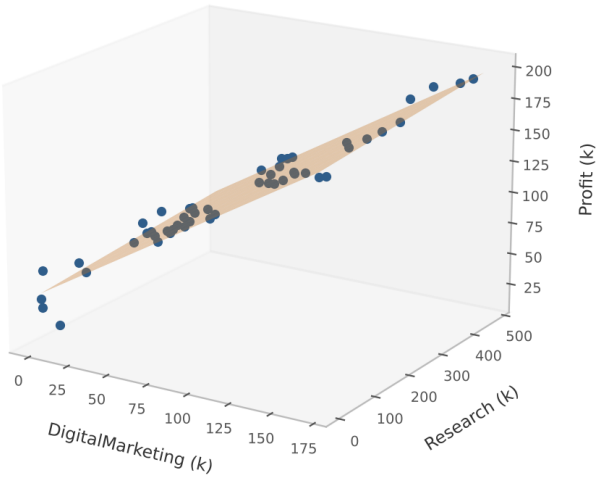


Figure 3. Profit against DigitalMarketing and Research (50 companies) with the fitted plane. Each dot's error is its vertical distance to the plane.

Part B. Building a model

4. The dataset: Investment.csv

The class data has **50 companies** and five columns, with no missing values. Profit is the dependent variable; the other four are the independent variables.

Column	Type	Meaning	Range or values
DigitalMarketing	number	Spend on digital marketing	0 to 165,349
Promotion	number	Spend on promotion	51,283 to 182,646
Research	number	Spend on research	0 to 471,784
State	category (text)	Where the company operates	Bangalore 17, Hyderabad 17, Chennai 16
Profit	number	**Dependent variable**	14,681 to 192,262

Look at the data before modelling. Profit rises with DigitalMarketing and Research and only weakly with Promotion.

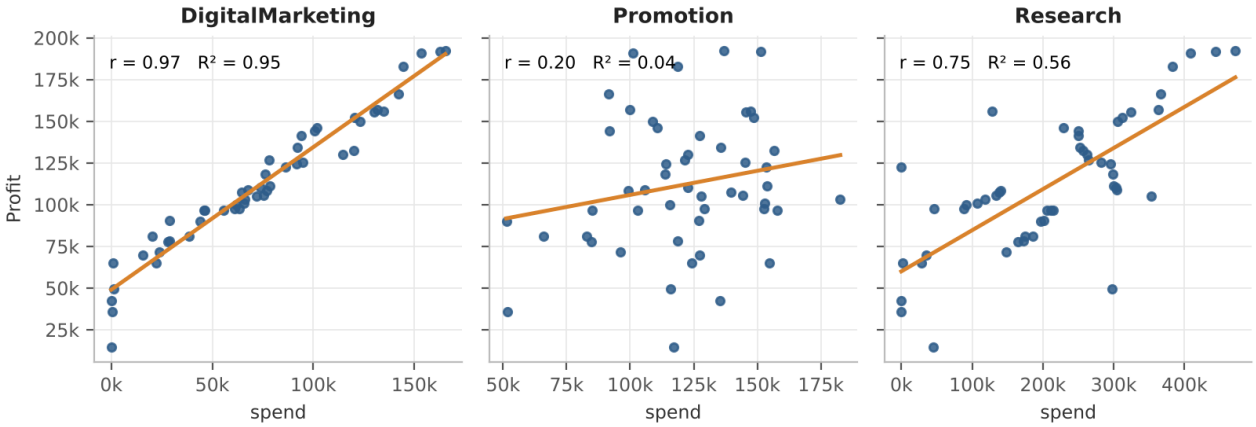


Figure 4. Profit against each spend, with the simple-regression line for each. Correlation with Profit: DigitalMarketing 0.97, Research 0.75, Promotion 0.20.

	DigitalMarketing	Promotion	Research	Profit
DigitalMarketing	1.00	0.24	0.72	0.97
Promotion	0.24	1.00	−0.03	0.20
Research	0.72	−0.03	1.00	0.75
Profit	0.97	0.20	0.75	1.00

The inputs are not independent of each other: DigitalMarketing and Research have a correlation of 0.72. That overlap is what Section 2 showed, and it returns in Section 10.

5. Categorical variables and dummy variables

A regression equation multiplies numbers, so a text column such as State cannot go in as it is. The standard fix is **one-hot encoding** (also called dummy variables): replace the column with one 0/1 column per category.

State	State_Bangalore	State_Chennai	State_Hyderabad
Bangalore	1	0	0
Chennai	0	1	0
Hyderabad	0	0	1

COMMON MISTAKE **The dummy variable trap**

The three columns always add up to 1 (every company is in exactly one state), and the model already has a constant column of 1s. So any one dummy can be computed from the others. This is **perfect multicollinearity**: the coefficients are no longer unique. Fix: keep **k − 1 dummies** for k categories.

You can see the problem in the numbers. With the constant plus all three dummies the design matrix has 7 columns but only **rank 6**, so one column is redundant. The category you leave out becomes the **reference category**, and every dummy coefficient is a difference from it.

Version	Code	State coefficients (training split)	Meaning
Class script (all 3 dummies)	`pd.get_dummies(X, dtype=int)`	Bangalore 86.64, Chennai −872.65, Hyderabad 786.01	Works in scikit-learn, but the three numbers are not individually meaningful
k − 1 dummies	`pd.get_dummies(X, dtype=int, drop_first=True)`	Chennai −959.28, Hyderabad 699.37	Bangalore is the reference: Chennai is this much below Bangalore, Hyderabad this much above, other inputs equal

KEY IDEA **Same predictions either way**

Both versions give **identical predictions** and the same R^2. Only the meaning of the State coefficients changes. Notice that Chennai − Bangalore = −872.65 − 86.64 = **−959.28** and Hyderabad − Bangalore = **699.37**: the all-dummies model hides the same two differences. scikit-learn and statsmodels will both still return numbers when the trap is present (they pick one of infinitely many equivalent solutions), but the individual coefficients are then arbitrary. Use `drop_first=True` whenever you want to interpret the coefficients.

6. Fitting the model in scikit-learn

The class workflow is: split the data, fit on the training rows, predict the test rows, then score. The code below is the class script written with `drop_first=True` and tidy names.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

data = pd.read_csv("Investment.csv")
X = data.iloc[:, :-1]                    # DigitalMarketing, Promotion, Research, State
y = data["Profit"]                       # dependent variable
X = pd.get_dummies(X, dtype=int, drop_first=True)   # State -> k-1 dummy columns

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0)             # 40 rows train, 10 rows test

model = LinearRegression().fit(X_train, y_train)
y_pred = model.predict(X_test)
print(model.coef_, model.intercept_)     # coefficients b1..bn and constant b0
print(model.score(X_train, y_train))     # training R-squared
print(model.score(X_test, y_test))       # test R-squared
```

Quantity	Value	How to read it
Training R^2	0.9502	95.0% of the variation in Profit is explained on the rows used to fit
Test R^2	0.9347	93.5% on 10 rows the model never saw. Slightly lower, as expected
Test RMSE	9,138	A typical prediction is off by roughly this much (in Profit units)
Test MAE	7,514	The average absolute error

The fitted equation on the training rows is:

$$\widehat{Profit} = 42,554.17 + 0.7735\,DM + 0.0329\,Promo + 0.0366\,Research - 959.28\,Chennai + 699.37\,Hyderabad$$

COMMON MISTAKE **Ten test rows is a small test set**

With only 10 test rows, the test R^2 depends heavily on *which* 10 rows landed there. Repeating the split with 50 different random seeds, the test R^2 had a median of **0.935** but ranged from **0.799 to 0.990**. One split (random_state=0) gives a reasonable estimate, not a precise one. Cross-validation (Section 9) is steadier.

7. How the coefficients are found

MLR uses the same rule as SLR: **least squares**. Choose b_0, b_1 … b_n to minimise the sum of squared errors.

$$SSE = \sum_{i=1}^{m} (y_i - \hat{y}_i)^2 = \sum_{i=1}^{m} (y_i - b_0 - b_1 x_{i1} - \cdots - b_n x_{in})^2$$

Setting the derivatives to zero gives the **normal equations**. In matrix form, where X has a column of 1s followed by one column per variable, they have the solution below. This is exactly what the library computes.

$$\mathbf{b} = (X^{\mathsf{T}}X)^{-1}X^{\mathsf{T}}\mathbf{y}$$

WORKED EXAMPLE **A five-row example you can check by hand**

The data below were built from the rule **y = 1 + 2·x₁ + 1·x₂**, so least squares must recover $b_0 = 1$, $b_1 = 2$, $b_2 = 1$.

Row	x_1	x_2	y
1	1	2	5
2	2	1	6
3	3	4	11
4	4	3	12
5	5	5	16

$X^{\mathsf{T}}X$	col 1	col 2	col 3		$X^{\mathsf{T}}y$		$\mathbf{b} = (X^{\mathsf{T}}X)^{-1}X^{\mathsf{T}}y$
row 1	5	15	15		50		$b_0 = 1$
row 2	15	55	53		178		$b_1 = 2$
row 3	15	53	55		176		$b_2 = 1$

Check: for a new row with $x_1 = 6$ and $x_2 = 3$ the model predicts $1 + 2\times6 + 1\times3 = \mathbf{16}$.

On the investment data, solving the normal equations by hand-coded linear algebra gives exactly the same six numbers as statsmodels (checked to 6 significant figures). If the columns of X are redundant (the dummy trap), X^TX has no inverse, which is why that case breaks the formula.

Part C. Reading the output and choosing variables

8. The statistical view: statsmodels

scikit-learn gives coefficients and scores. To judge **which coefficients are trustworthy**, use statsmodels, which also reports standard errors, t-tests and the F-test. One thing to know: `sm.OLS` does **not** add a constant by itself. Add it with `sm.add_constant`.

```
import statsmodels.api as sm

X = pd.get_dummies(data.iloc[:, :-1], dtype=int, drop_first=True).astype(float)
X = sm.add_constant(X)              # adds the column of 1s for b0
ols = sm.OLS(y, X).fit()
print(ols.summary())
```

Term	Coefficient	Std. error	t	p-value
const (b_0)	50,125.34	6,884.82	7.28	< 0.001
DigitalMarketing	0.8060	0.0464	17.37	< 0.001
Promotion	−0.0270	0.0522	−0.52	0.608
Research	0.0270	0.0171	1.57	0.123
State_Chennai	198.79	3,371.01	0.06	0.953
State_Hyderabad	−41.8870	3,256.04	−0.01	0.990

This table uses all 50 rows with Bangalore as the reference. Overall: R^2 = **0.9508**, adjusted R^2 = **0.9452**, F = **169.89** (p = 1.3e-27).

Item	Question it answers	Here
t-test for a coefficient	Is this coefficient different from 0 **given the other variables in the model**? (H_0: coefficient = 0)	Only DigitalMarketing is clearly non-zero ($p < 0.001$). Promotion, Research and the State dummies are not ($p > 0.05$)
F-test	Is **at least one** slope non-zero? (H_0: all slopes = 0)	Yes: F = 169.89, p ≈ 1e-27
R^2	Share of the variation in y explained	0.9508
Adjusted R^2	R^2 corrected for the number of predictors	0.9452

Adjusted R^2

R^2 never goes down when you add a variable, even a useless one, so it cannot be used to compare models with different numbers of predictors. **Adjusted R^2** penalises each extra predictor. With n rows and p predictors:

$$R^2_{adj} = 1 - (1 - R^2)\frac{n-1}{n-p-1}$$

Check with the full model: n = 50, p = 5, $R^2 = 0.9508$, so adjusted $R^2 = 1 - 0.0492 \times 49/44 =$ **0.9452**, matching the table.

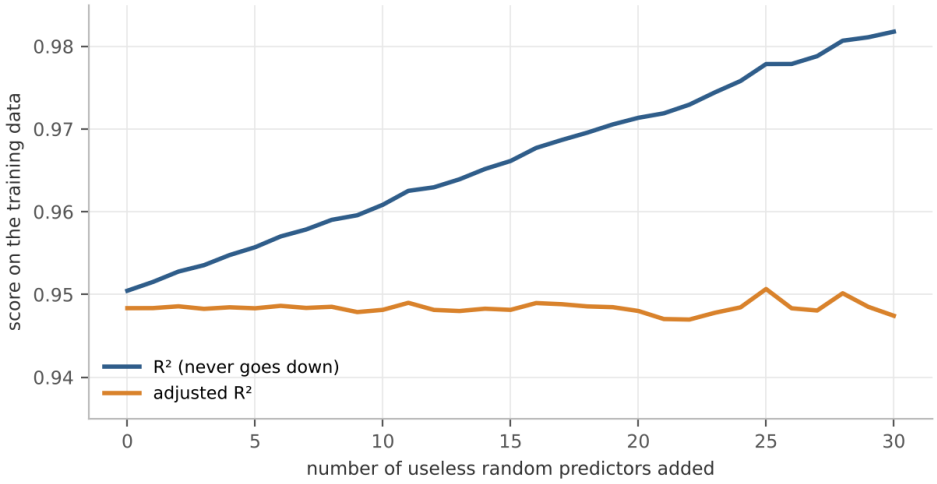


Figure 5. Adding random, meaningless predictors to a model with DigitalMarketing and Research (average of 150 trials per point). R^2 climbs from 0.950 to 0.982; adjusted R^2 does not improve.

> **NOTE** **A note on the class script's constant column**
>
> The class script builds its own constant with `np.full((50, 1), 42467)`, the rounded scikit-learn intercept, instead of a column of 1s. That is not wrong, because any constant column acts as the intercept, but the coefficient on it is then **1.1794** and the real intercept is 1.1794 × 42,467 = **50,083.46**. P-values and R^2 are unaffected. `sm.add_constant` avoids the extra step.

9. Choosing variables: backward elimination

Not every column helps. Extra predictors add noise, make the model harder to explain, and cost data. **Backward elimination** is the method used in class:

1. Pick a significance level to stay, usually **SL = 0.05**.
2. Fit the **full model** with all predictors.
3. Find the predictor with the **highest p-value**.
4. If that p-value is **above SL**, remove the predictor, refit, and go back to step 3.
5. If it is **at or below SL**, stop. The remaining predictors are the final model.

A removed variable never comes back (that is what separates it from stepwise selection).

Step	Remove	Its p-value	R^2 after	Adjusted R^2 after
1	State: Hyderabad	0.990	0.9508	0.9464
2	State: Chennai	0.940	0.9507	0.9475
3	Promotion	0.602	0.9505	0.9483
4	Research	0.060	0.9465	0.9454
Stop	nothing	DigitalMarketing p < 0.001	0.9465	0.9454

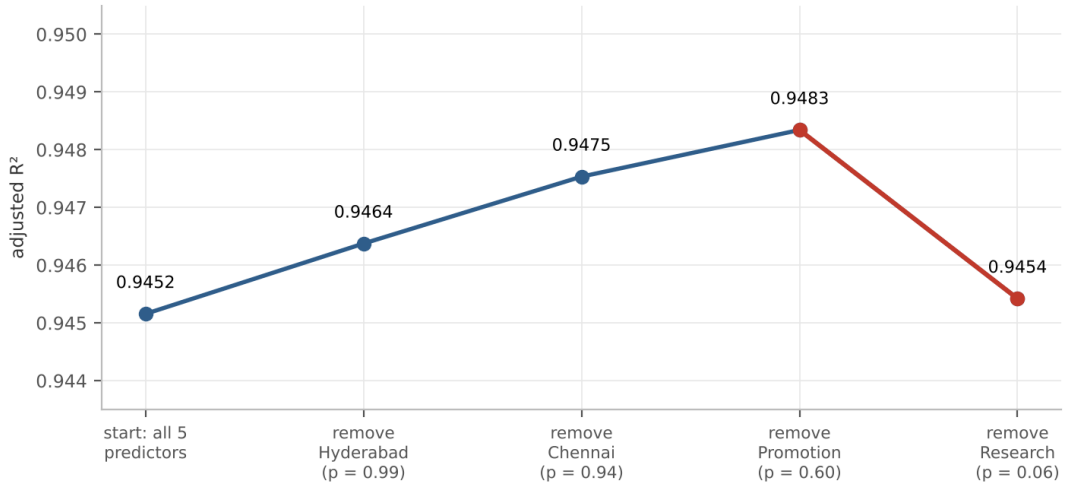


Figure 6. Adjusted R^2 along the backward-elimination path. It rises while the useless variables are removed and falls only at the last step (Research).

The method ends with one predictor: **Profit = 49,032.90 + 0.8543 × DigitalMarketing**, R^2 = 0.9465. Notice how little is lost: R^2 goes from 0.9508 to 0.9465 while four predictors disappear.

COMMON MISTAKE **Research was a close call, and the rules disagree**

Research was removed with p = **0.060**, barely above 0.05, and removing it **lowers** adjusted R^2 (0.9483 → 0.9454). 5-fold cross-validation (R^2, shuffled, random_state=0) says: all variables **0.9179**, DigitalMarketing only **0.9294**, DigitalMarketing + Research **0.9333**. So p-values, adjusted R^2 and cross-validation do not give one answer. 0.05 is a convention, not a law; with 50 rows, treat Research as undecided and ask whether it makes business sense.

KEY IDEA **Cautions with p-value selection**

- The p-values describe the **training data**. Selecting variables by them and then reporting the same data's p-values is optimistic.
- Correlated predictors share credit, so a useful variable can look useless (Research here) until its partner is removed.
- Prefer a **held-out test set or cross-validation** to decide, and use p-values as a guide.
- scikit-learn has no p-values; this is why the class switches to statsmodels for this step.

A short, well-commented way to automate the loop (the class does it by hand with column indices):

```
def backward_elimination(X, y, sl=0.05):
    X = sm.add_constant(X.astype(float))
    while True:
        model = sm.OLS(y, X).fit()
        worst = model.pvalues.drop("const").idxmax()   # highest p-value
        if model.pvalues[worst] <= sl:                 # all remaining are significant
            return model
        X = X.drop(columns=worst)                      # remove it and refit
```

10. Assumptions and multicollinearity

MLR inherits the SLR assumptions and adds one: **the predictors should not be (near-)copies of each other.**

Assumption	What it means	How to check	Investment data
Linearity	Profit changes linearly with each x	Scatter plots, residuals vs fitted	Good: Figure 4 and no curve in Figure 7
Independence	Errors are unrelated to each other	Durbin-Watson (about 2 is ideal)	1.28: some positive autocorrelation. The rows have no time order, so this is a mild concern
Constant spread	Same error spread at all fitted values	Residuals vs fitted	Roughly even
Normal errors	Errors roughly bell-shaped	Histogram, Q-Q plot, Shapiro test	Mildly off (Shapiro p = 0.011); a few low-profit rows pull the left tail
No multicollinearity	Predictors are not strongly related to each other	Correlation matrix, VIF	Fine: all VIFs below 2.5

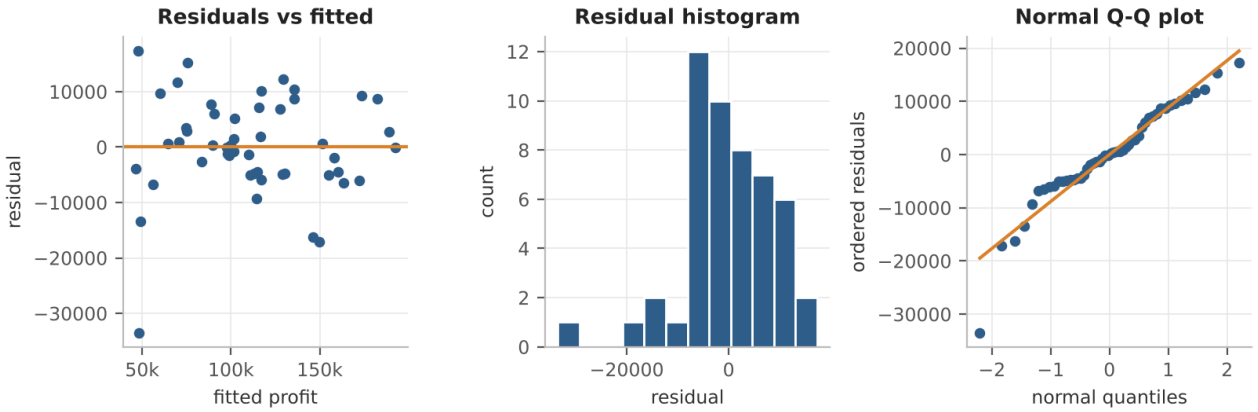


Figure 7. Residual diagnostics for the full model. The most negative residual (−33,504) belongs to a company that spent 0 on DigitalMarketing.

Multicollinearity and VIF

DEFINITION **Multicollinearity**

Two or more independent variables are strongly related to each other. The model can still predict well, but the individual coefficients become unstable and hard to interpret: their standard errors grow and signs can flip.

The **variance inflation factor (VIF)** measures this. For predictor j, regress it on all the other predictors and take that regression's R^2:

$$VIF_j = \frac{1}{1 - R_j^2}$$

Penn State's STAT 462 guideline: a VIF above **4** is worth investigating and above **10** signals serious multicollinearity.

Predictor	VIF	Verdict
DigitalMarketing	2.469	Below 4: fine
Promotion	1.175	Below 4: fine
Research	2.327	Below 4: fine

KEY IDEA **Perfect multicollinearity**

When one variable is an exact combination of others (like the dummy trap), the coefficients cannot be determined at all. Section 11 shows it in a real dataset.

11. A second example: house prices

The class notebook `MLR-Housing.ipynb` predicts house price from 18 columns for 21,613 sales (after dropping `id` and `date`). The mean price is 540,088.

Quantity	Value
Split	1/3 test, random_state=0 (14,408 train, 7,205 test)
Training R^2	0.7061
Test R^2	0.6856
Test RMSE	213,041
Test MAE	123,805
Full OLS adjusted R^2	0.6995
Backward elimination at 0.05	removes only `floors` ($p = 0.063$); adjusted R^2 barely moves (0.69951 → 0.69948)

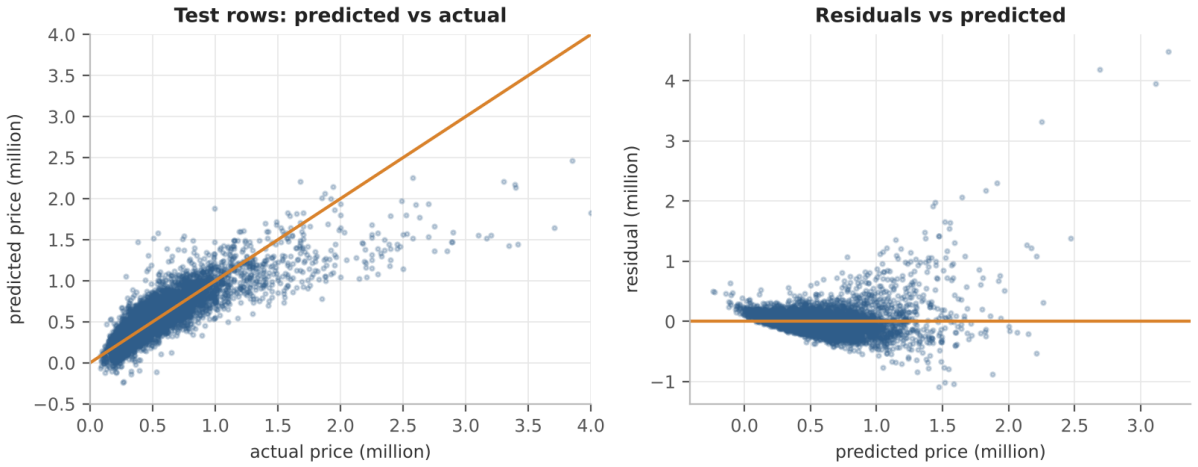


Figure 8. House-price model on the test rows. The fit is good for typical prices and loses accuracy for the most expensive houses; residuals fan out as the predicted price rises.

COMMON MISTAKE **A hidden problem: sqft_living = sqft_above + sqft_basement**

In **every one of the 21,613 rows**, `sqft_living` equals `sqft_above` plus `sqft_basement`. That is perfect multicollinearity, so the 18 predictor columns have only **rank 17**, and statsmodels warns that the parameters are not uniquely determined. Predictions are still fine (dropping `sqft_basement` leaves the test R^2 unchanged at 0.6856), but the three area coefficients **cannot be interpreted individually**. Remove one of the three.

- `zipcode` is stored as a number, but it is a **category** (a bigger zipcode is not 'more' of anything). Treat it as a categorical variable or leave it out.
- The notebook uses `sklearn.cross_validation`, which has been removed. Use `sklearn.model_selection.train_test_split`.
- The notebook hard-codes `np.zeros((21613, 19))`. Use `sm.add_constant(X)` so it works for any number of rows.

- Price is skewed (mean above median), which is why residuals widen at high prices. Modelling the logarithm of price is a common fix.

Part D. Wrap-up

12. Common mistakes

Mistake	Why it is a problem	Do this instead
Reading a coefficient as 'the effect of x'	It is the effect with the others held fixed, and it changes when the model changes	Say 'holding the other variables constant'
Encoding State as 1, 2, 3	Implies Hyderabad is 'twice' Bangalore	Use dummy variables
All k dummies plus a constant	Dummy variable trap (perfect multicollinearity)	Use k − 1 dummies (`drop_first=True`)
Comparing models by R^2	R^2 rises with every added variable	Use adjusted R^2 or cross-validation
`sm.OLS` without a constant	Forces the line through the origin	Use `sm.add_constant`
Dropping a variable only because p is 0.06	p-values are noisy with small n, and correlated variables share credit	Check adjusted R^2, cross-validation and domain sense
Judging the model on 10 test rows once	The score changes a lot between splits	Repeat the split or cross-validate
Comparing coefficient sizes across different units	A larger number may just mean smaller units	Standardise the variables first
Ignoring redundant columns	Coefficients become meaningless	Check VIF and obvious sums such as living = above + basement

Summary

SUMMARY

- **MLR** predicts one numerical dependent variable from two or more independent variables: $\hat{y} = b_0 + b_1x_1 + \ldots + b_nx_n$.
- Each coefficient is the effect of one x **holding the others constant**, so it can change when variables are added or removed (Research: 0.246 alone, 0.027 in the full model).
- Coefficients come from **least squares**, $b = (X^TX)^{-1}X^Ty$.
- Turn categories into **k − 1 dummy variables**; the omitted one is the reference.
- R^2 never falls when variables are added; **adjusted R^2** penalises extras.
- **Backward elimination** removes the highest p-value above 0.05, one at a time. On Investment it ends with DigitalMarketing only, but Research was borderline and the other measures disagree.
- Check linearity, independence, constant spread, normal errors and **multicollinearity (VIF)**; exact redundancy (as in the housing data) makes coefficients meaningless.

Practice questions

Try each question before reading the answer underneath it.

Q1. In the full Investment model, the coefficient of DigitalMarketing is 0.806. Explain it in one sentence.

> **Answer.** Holding Promotion, Research and State fixed, one extra unit of DigitalMarketing spend goes with about 0.806 more units of Profit on average.

Q2. A column has 5 categories. How many dummy columns do you keep when the model has a constant, and why?

> **Answer.** Four. With all five plus a constant the dummies add up to the constant column (the dummy variable trap), so the coefficients would not be unique.

Q3. A model with n = 50 rows and p = 5 predictors has $R^2 = 0.9508$. Compute the adjusted R^2.

> **Answer.** $1 - (1 - 0.9508) \times 49/44 =$ **0.9452**.

Q4. Using b = (1, 2, 1) from Section 7, predict y when $x_1 = 6$ and $x_2 = 3$.

> **Answer.** $1 + 2 \times 6 + 1 \times 3 =$ **16**.

Q5. Research has p = 0.060 in the last backward-elimination step. Should it be removed at SL = 0.05?

> **Answer.** By the rule, yes (0.06 > 0.05). But adjusted R^2 drops (0.9483 → 0.9454) and cross-validated R^2 is higher with Research (0.9333 vs 0.9294), so it is a judgement call, not a clear-cut removal.

Q6. A predictor has VIF = 5. What R^2 would you get regressing it on the other predictors, and is that a concern?

> **Answer.** $VIF = 1/(1 - R^2) = 5$, so $R^2 = 0.8$. It is above the guideline of 4, so investigate whether it overlaps heavily with another predictor.

Q7. The design matrix with the constant and all three State dummies has 7 columns. What is its rank and what does that mean?

> **Answer.** Rank 6, one less than the number of columns: one column is redundant (the dummy variable trap), so one dummy must be dropped.

Q8. Why does the housing model have unreliable individual area coefficients even though its predictions are fine?

> **Answer.** `sqft_living` = `sqft_above` + `sqft_basement` in every row, so the columns are perfectly collinear. Many coefficient sets give the same predictions, so none of them can be interpreted on its own.

Glossary

Term	Meaning
Multiple Linear Regression (MLR)	Linear regression with two or more independent variables and one dependent variable
Dependent variable (DV)	The value being predicted (target, y)
Independent variable (IV)	An input used for prediction (feature, predictor, x)
Constant (intercept, b_0)	The predicted y when every x is 0
Coefficient (b_1 … b_n)	Change in predicted y for one more unit of that x, others held constant
Dummy variable	A 0/1 column that marks one category
Reference category	The category left out when using k − 1 dummies; other dummies are measured against it
Dummy variable trap	Keeping all k dummies with a constant, which makes the columns perfectly redundant
Least squares (OLS)	Choosing coefficients that minimise the sum of squared residuals
Normal equations	The equations $X^TX\ b = X^Ty$ that least squares solves
p-value	Probability of a result at least this extreme if the true coefficient were 0
F-test	Tests whether all slope coefficients are 0 together
Adjusted R^2	R^2 corrected for the number of predictors
Backward elimination	Start with all predictors and repeatedly drop the least significant one
Multicollinearity	Predictors that are strongly related to each other
VIF	Variance inflation factor, $1/(1 - R_j^2)$, a measure of multicollinearity
Cross-validation	Repeating the train/test split several ways and averaging the score

Sources

- Penn State STAT 462, *Applied Regression Analysis*: the multiple linear regression model and coefficient interpretation (online.stat.psu.edu/stat462) and the VIF guidelines (4 and 10).
- pandas documentation, `pandas.get_dummies` (`drop_first` returns k − 1 dummies).
- MathWorks documentation, *Coefficient of determination (R-squared)* and adjusted R-squared.
- Minitab documentation, *Stepwise and best subsets regression methods* (backward elimination).
- scikit-learn user guide, *Linear Models*; statsmodels documentation, *OLS* and `add_constant`.
- Class slides *Multiple Linear Regression*, `class practise.py`, `Investment.csv`, `House_data.csv` and `MLR-Housing.ipynb`.
- Every number in these notes was recomputed with pandas, scikit-learn, statsmodels and SciPy from the class files.